Design Messenger

功能性需求（Functional Requirements）

1. 使用者（Users）應該能夠建立包含多位參與者的群組聊天（group chats），人數上限為 100 人。
2. 使用者應該能夠傳送與接收訊息（send/receive messages）。
3. 使用者在離線狀態下，仍應該能夠接收其他人於期間送出的訊息。

不在範圍內（Out of Scope）

1. 傳送附件（send attachments）
2. 視訊通話（video calling）
3. 線上狀態顯示（online presence）

非功能性需求（Non-Functional Requirements）

1. 訊息應以低延遲（low latency）傳遞給在線使用者，延遲需低於 500ms。
2. 耐久性（Durability）：使用者送出的訊息不可遺失。
3. 一致性（Consistency）：同一個聊天中的所有使用者，必須看到完全一致的訊息順序（message order）。
4. 系統需能支援 1 億（100M）使用者，並具備高吞吐量（high throughput）。

網路通訊協定（Network Protocols）

在即時通訊應用中，當一位使用者送出訊息時，同一個聊天中的其他使用者必須在毫秒級時間內看到該訊息。

在這類應用中，不可能讓每個使用者每隔幾毫秒就不斷向 server polling 更新，否則基礎設施會立刻被壓垮。

buildmoat.org

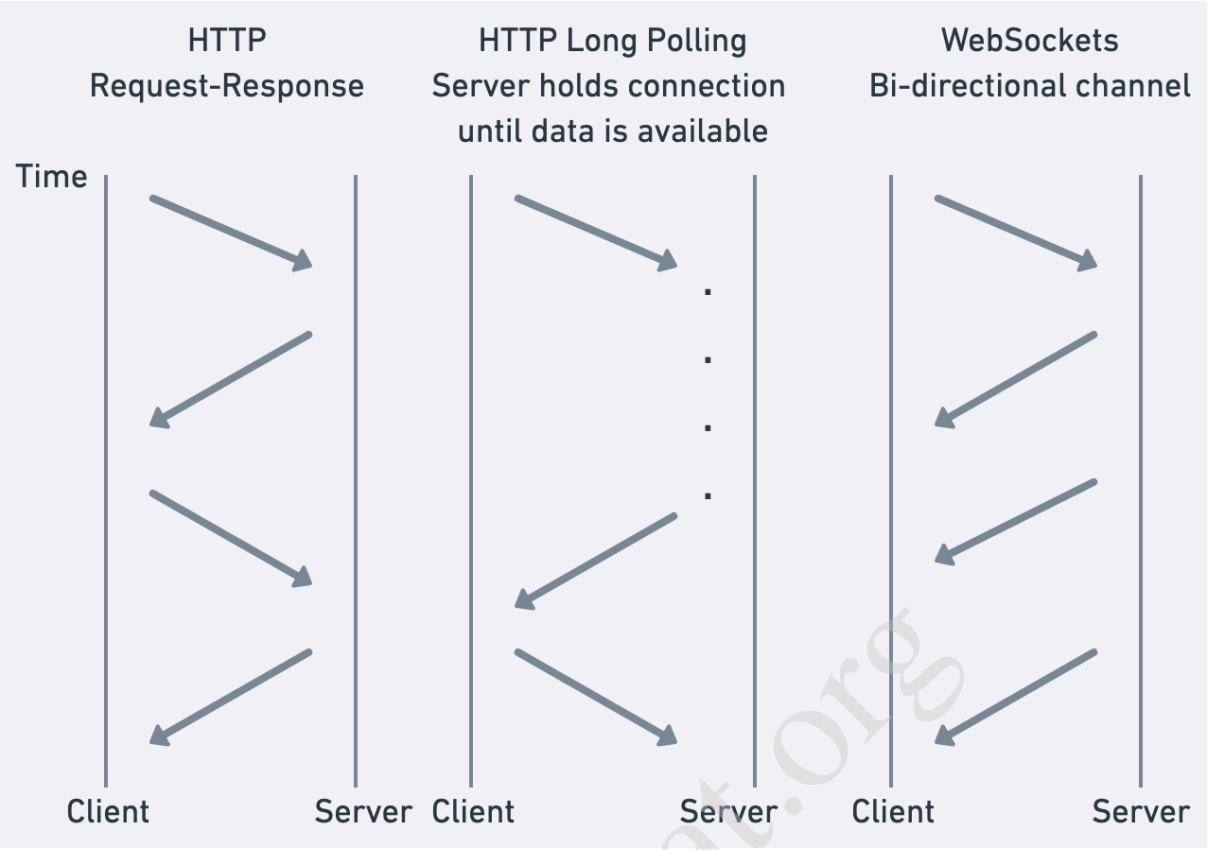
核心挑戰在於：如何在 client 與 server 之間建立**高效率、可長時間維持的通訊通道**。

標準 HTTP 採用的是 request-response model：client 發送請求，server 回應後連線即關閉。

這對傳統網頁瀏覽非常合適，但當 server 需要**主動推送（push）更新**給 client 時，就完全不適用了。

協定方式	機制說明	優點	缺點	適用情境
HTTP 簡單輪詢 (Simple Polling)	客戶端固定時間間隔發送請求，伺服器立即回應（無資料則回傳空結果）。	實作簡單、相容性佳	- 大量空回應造成資源浪費- 高延遲，無法即時	**低頻率更新**，如偶爾檢查通知
HTTP 長輪詢 (Long Polling)	客戶端發送請求，伺服器保持請求直到有新資料才回應。回應後客戶端立即發出新請求。	減少空回應，比簡單輪詢更即時	- 每次回應後仍需重建連線- 頻繁更新時延遲仍存在- 保持連線會造成資源壓力	**中低頻即時需求**，如簡單聊天或通知系統
WebSocket	基於 HTTP Upgrade 建立持久連線，雙向即時傳輸。	- 真正雙向通訊- 高頻低延遲- 傳輸效率高（可傳 JSON/Protobuf 等格式）	- 較複雜的實作與維護- 需處理連線管理與錯誤恢復	**高頻即時通訊**，如聊天、遊戲、即時協作

buildnotes



HTTP 簡單輪詢（HTTP Simple Polling）

Client 會重複向 server 發送請求以取得新資料。

Client 發送 request，等待 server 回應；若目前沒有新資料，server 則回傳空回應。

問題（Problem）：

Client 必須持續詢問 server 是否有新資料，導致大量空回應，產生不必要的 HTTP 開銷（overhead）。

HTTP 長輪詢（HTTP Long Polling）

Client 向 server 發送 request，而 server 會**保持連線不回應**，直到有新資料可回傳。

這看起來就像是 server 花了很長時間才處理完請求。

一旦 server 回傳資料並結束該 HTTP request，client 會立刻再送出一個新的 request，如此反覆。

問題（Problem）：

- 在更新頻繁的情況下，client 無法即時收到資料，因為每次接收完資料後，還需要再發送一次新的 request。
- 即使使用者很少聊天，client 仍必須定期與 server 建立連線，造成整體效率不佳。

buildmoat.org

WebSocket

WebSocket 是 client 與 server 之間**真正雙向（bi-directional）通訊**的首選方案。

在需要高頻讀寫（high frequency writes and reads）的場景中，WebSocket 幾乎是最佳解。

WebSocket 是基於 HTTP 的 **upgrade protocol**，允許既有的 TCP 連線在第七層（L7）切換通訊協定。

一旦連線建立，client 與 server 便可互相傳送「訊息」，這些訊息本質上是 opaque binary blobs，可以承載字串、JSON、Protobuf，或任何其他格式。

WebSocket 連線流程如下：

1. Client 透過 HTTP 發起 WebSocket handshake
2. 連線升級為 WebSocket protocol
3. Client 與 server 皆可隨時傳送訊息
4. 連線會持續存在，直到其中一方明確關閉

API 設計（API Design）

由於我們使用的是 WebSocket（雙向即時通訊協定）連線，API 不需要遵循 REST 慣例。

為了簡化說明，這裡使用 JSON 作為資料格式。

1. 建立聊天（createChat）

```
{
  "participants": [],
  "name": ""
} -> {
  "chatId": ""
}
```

- `participants`（參與者清單）：聊天室中所有成員的 user IDs。
- `name`（聊天室名稱）：群組聊天的顯示名稱。
- `chatId`（聊天室 ID）：系統建立並回傳的唯一識別碼。

buildmoat.org

2. 傳送訊息（sendMessage）

```
{
  "chatId": "",
  "message": ""
} -> "SUCCESS" | "FAILURE"
```

- `chatId`：目標聊天室的識別碼。
- `message`（訊息內容）：使用者要傳送的文字訊息。
- 回傳結果表示訊息是否成功送出。

3. 修改聊天室成員（modifyChatParticipants）

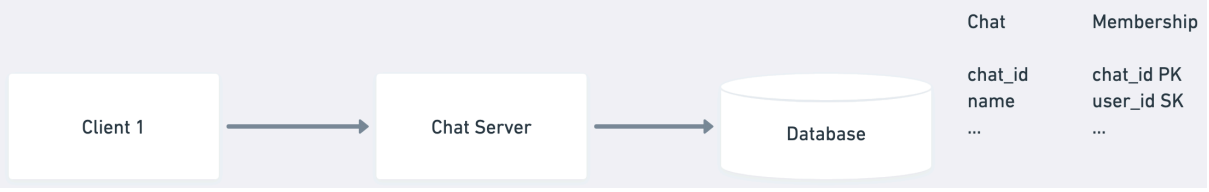
```
{
  "chatId": "",
  "userId": "",
  "operation": "ADD" |"REMOVE"
} -> "SUCCESS" | "FAILURE"
```

- `chatId`：要修改的聊天室。
- `userId`（使用者 ID）：要加入或移除的成員。
- `operation`（操作類型）：
  - `ADD`：新增成員
  - `REMOVE`：移除成員

buildmoat.org

High-Level Design

1. 使用者應該能夠建立包含多位參與者的群組聊天



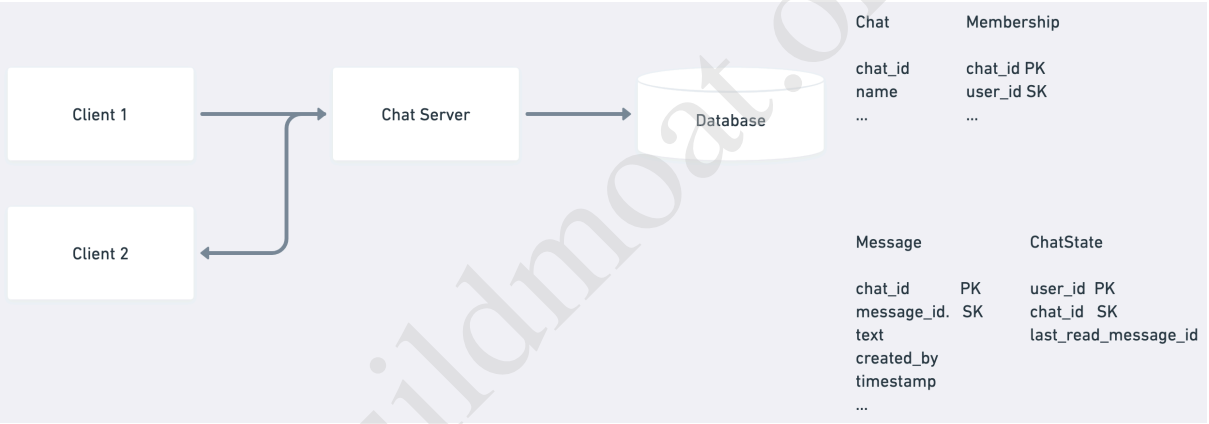
1. 使用者連線至 chat server，並送出 `createChat` 訊息。

2. Server 建立一筆 `chat` 紀錄，並為每位參與者建立一筆 `membership` 紀錄；這些寫入操作會包在同一個 DB transaction 中。
3. Server 將產生的 `chatId` 回傳給使用者。
4. 由於 Database 會承受大量寫入吞吐量（write throughput），我們可以使用 NoSQL（例如 DynamoDB）來儲存資料。
5. 為了能有效率地查詢同一個聊天中的所有參與者，我們使用 `chat_id` 作為 partition key，`user_id` 作為 sorting key。

2. 訊息傳遞流程（Message Delivery Flow）

為了簡化說明，先假設所有使用者都連線到同一台 server，且我們使用一個 local hash map 來維護 `user_id → WebSocket connection` 的對應關係。

如何擴展到數億使用者，會在 deep dive 中再討論。



當接收者在線（recipient is online）：

1. 使用者向 Chat Server 傳送 `sendMessage` 訊息。
2. Chat Server 將訊息寫入 `Message` table。
   a. `message_id` 設計為 monotonic increasing（單調遞增），以確保訊息能依時間順序排序；此設計會在 deep dive 中進一步說明。
3. Chat Server 透過 `Membership` table 查詢該聊天室中的所有參與者。
4. Chat Server 將最終的 `message_id` 與 `SUCCESS` 或 `FAILURE` 回傳給發送者。
5. Chat Server 依序查找每位參與者對應的 WebSocket connection，並透過 `newMessage` 將訊息推送給他們。
6. Client 在收到訊息後，會送出 `ack` 訊息給 Chat Server，表示訊息已成功接收；Chat Server 接著會更新 `Membership` table 中的 `last_read_message_id`。

buildmoat.org

當接收者離線（recipient is offline）：

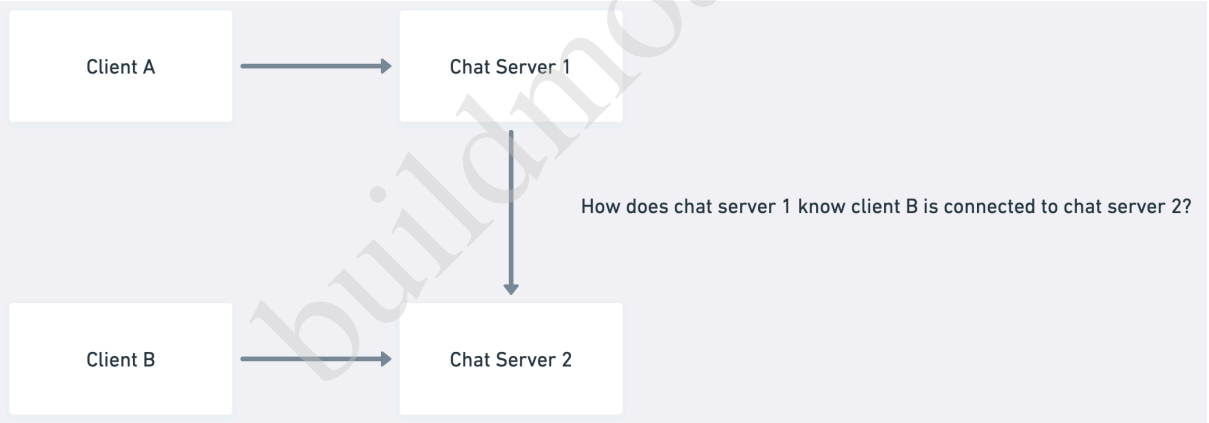
1. 查詢 ChatState table，取得該使用者的 `last_read_message_id`。
2. 從 Message table 中查詢所有 `message_id` > `last_read_message_id` 的訊息。
3. Chat Server 將這些未讀訊息推送給接收者。
4. Client 收到訊息後，會送出 `ack` 訊息給 Chat Server。
5. Chat Server 更新 `Membership` table 中的 `last_read_message_id`。

深入探討（Deep Dives）

1. 我們要如何處理數億使用者同時在線？

我們需要部署多台 server（可能是數百台；WhatsApp 單一 host 可承載約 100–200 萬使用者）。

此處最關鍵的問題是：**當發送者與接收者連線到不同的 hosts 時，如何定位接收者所在的 host，以便發送者能即時推送訊息？**



1. 以使用者歸屬為基礎的 Consistent hashing of Chat servers

（Consistent hashing of Chat servers of user ownership）

Note：Consistent hashing 是用來解決 partition 問題的基礎演算法，目標在於：

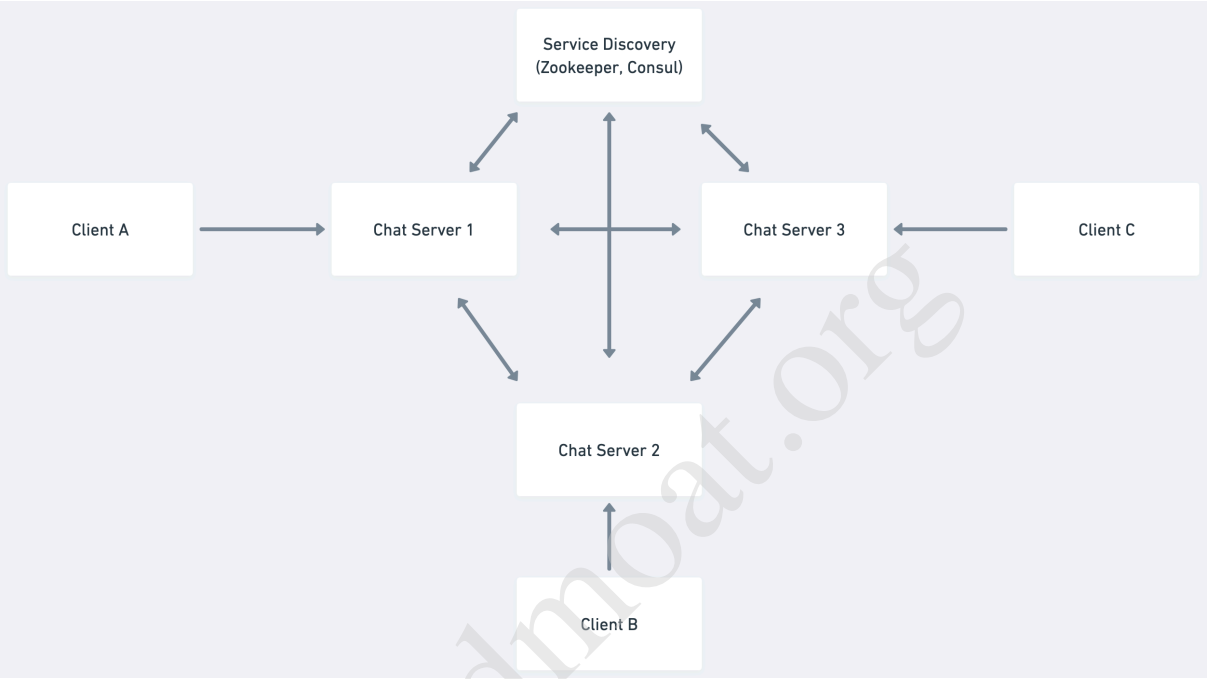
1. 將目標（connections、data）平均分配到可用節點
2. 在新增或移除節點時，將影響降到最低

我們可以讓每台 server 負責一段 `user_id` 的 key space，將使用者連線平均分散到各台 server。

要根據 `user_id` 定位對應的 server，可使用集中式、強一致性的方式：

buildmoat.org

- **集中式 Service Discovery**（例如 Zookeeper、Consul）
  - 以 Raft 或 Zab 共識為基礎的強一致性儲存系統
  - 所有節點都會註冊到 service registry
  - Registry 維護 `user_id → node` 的對應關係，並定期更新給各節點
  - 每個節點會將這份對應表快取在本地，以加速查詢

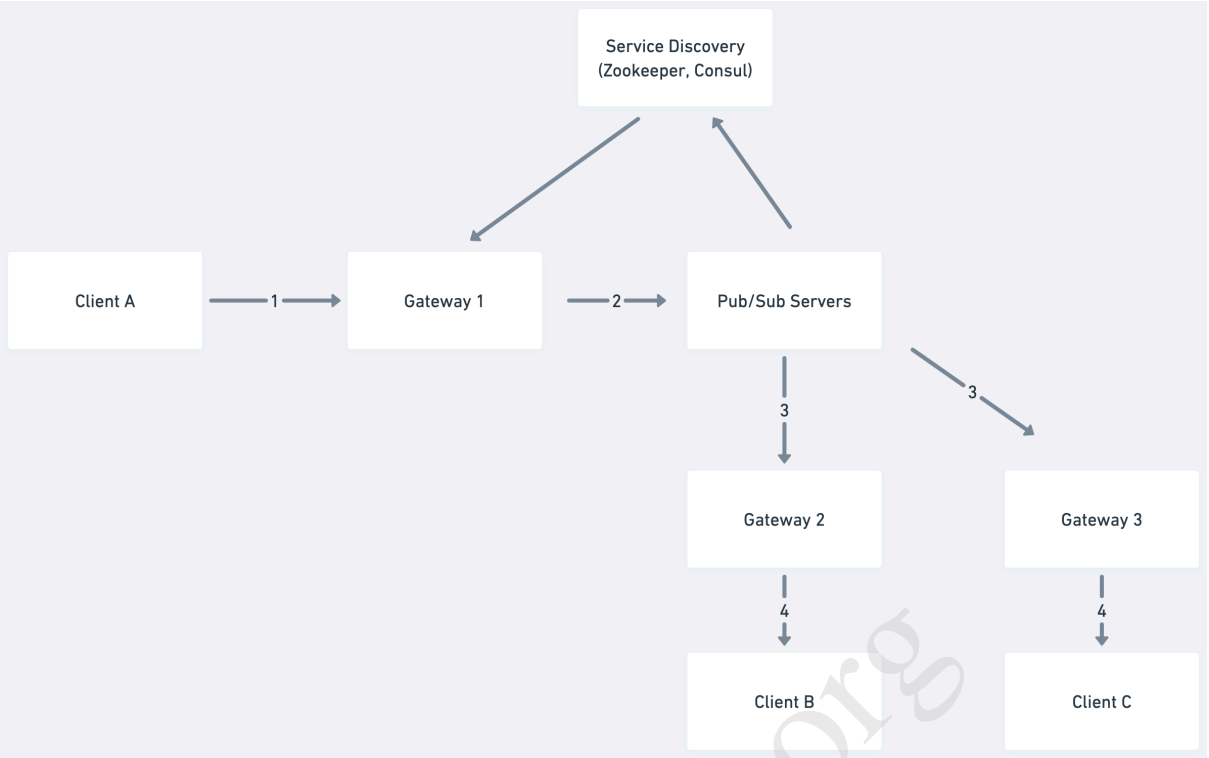


這個方式在理論上可行，特別是一對一聊天的情境下；但當使用者規模成長後，系統會變得難以維護：

1. 每一則訊息都需要發送 N 次 RPC；對大型群聊而言，每條訊息可能需要數百到上千個 RPC。
2. 系統進行 scale up / scale down 時，除了 socket connection，還必須重新分配 `user_id` 的 ownership，操作非常困難。
3. 更根本的問題在於：以 user 為分片單位時，同一個 chat 會分散在多個 chat server owners 上（A 發的訊息在 server 1，B 發的訊息在 server 2），幾乎無法強制維持單一的訊息順序。

2. Pub/sub model based on chat

buildmoat.org



與其以 `user_id` 為核心，這個方案改以 `chat_id` 為核心，並將 socket connection 與 chat ownership 解耦，設計如下：

1. Key space 以 `chat_id` 組成，service registry 維護 `shard_id → pub/sub server address` 的對應關係。
    a. `shard_id = hash(chat_id) % S`，其中 S 為 shard 數量。
2. 每個 pub/sub server 在記憶體中維護 `chat_id → [subscribing_gateways]` 的對應表。
3. Pub/sub server 透過 heartbeat 向 service discovery 回報節點存活狀態與 shard ownership。
4. Gateway 啟動時，從 service discovery 載入 shard mapping，並快取在本地。
    a. Gateway 也會定期監聽 service discovery 的更新。
5. 當使用者連線至 gateway 並加入某個 chat，gateway 會向該 chat 的 owning pub/sub server 發送 `Subscribe(chat_id)`；owner 會將該 gateway 加入 `subscribers[chat_id]`。
6. 訊息傳遞流程為：

    sender → gateway → pub/sub → subscribed gateways → receiver

依此 pub/sub model：

buildmoat.org

1. 每則訊息不再需要對 N 位接收者做 fan-out，只需對 M 個 subscribing gateways 做 fan-out。
2. Gateway 變為 stateless，使用者可連線至任意 gateway，水平擴展更容易。
3. 最重要的是：每個 chat 都有唯一的 pub/sub server owner，讓強制訊息順序一致性成為可行方案（下一節將說明）。

2. 我們要如何處理訊息亂序（out-of-order message delivery）？

設想一個情境：同一個 chat 中，有兩位使用者同時發送訊息，如何確保最終所有人看到的訊息順序一致？

處理亂序最常見的方式是：**讓接收並持久化訊息的 server 成為訊息順序的唯一真實來源（source of truth）**。

前述的 pub/sub 架構正好具備這個特性：

同一個 chat 的所有訊息都會被路由到同一個 pub/sub server，因此我們可以在該 server 上強制訊息順序，並將訊息持久化到 DB。

當 pub/sub server 接收到並處理訊息後，會指派一個 `message_id`，且該 ID 為 monotonic increasing（單調遞增）。

我們不能使用 client 端產生的 `timestamp`，原因包括：

- 不同 client 之間可能存在 clock skew
- Timestamp 本身不保證唯一性（多個 client 可能在同一毫秒產生訊息）

常見的 monotonic ID 產生方式包括：

- 每個 partition 使用 auto-increment counter
- **Timestamp-based UUIDs（UUID v1）**
- Twitter Snowflake IDs

這類 monotonic increasing ID（例如 auto-increment number 或 UUID v1）同時具備**時間順序性**與**唯一性**。

在 client 端，當收到訊息時，若出現 late-arriving message（`message_id` 較小），client 會在本地重新排序對話內容，以確保呈現順序正確。

buildmoat.org